

final_homework

January 12, 2026

```
[1]: import trino
      from trino.dbapi import connect
      import pandas as pd
      import matplotlib.pyplot as plt
      import seaborn as sns
      import psycopg as pg
      import pymysql
```

```
[7]: %pip install psycopg
```

```
Collecting psycopg
  Downloading psycopg-3.3.2-py3-none-any.whl.metadata (4.3 kB)
Requirement already satisfied: tzdata in c:\users\daniel\anaconda3\lib\site-packages (from psycopg) (2025.2)
Downloading psycopg-3.3.2-py3-none-any.whl (212 kB)
Installing collected packages: psycopg
Successfully installed psycopg-3.3.2
Note: you may need to restart the kernel to use updated packages.
```

```
[27]: %pip install PyMySQL
```

```
Collecting PyMySQL
  Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
Downloading pymysql-1.1.2-py3-none-any.whl (45 kB)
Installing collected packages: PyMySQL
Successfully installed PyMySQL-1.1.2
Note: you may need to restart the kernel to use updated packages.
```

```
[5]: #           ,           , etc

iceberg_bucket = "local/iceberg"

trino_creds = dict(
    host = "localhost",
    port = 8080,
    user = "jupyter"
)

iceberg_creds = trino_creds.copy()
```

```

iceberg_creds.update(
    dict(
        catalog="iceberg",
        schema="student_superstore"
    )
)

trino_creds_upd = trino_creds.copy()
trino_creds_upd.update(
    dict(
        catalog="system",
        schema="runtime",
    )
)

pg_creds = dict(
    host='localhost',
    port=5432,
    user='ps_user',
    password='ps_pass',
    dbname='demo_db'
)

mysql_creds = dict(
    host="localhost",
    port=3306,
    user="mysql_user",
    passwd="mysql_pass",
    db="demo_db",
    charset="utf8mb4",
)

```

```

[3]: # kaggle -- https://www.kaggle.com/datasets/konstantinognev/
      ↪sample-superstorecsv
      # pg mysql

df = pd.read_csv("Sample - Superstore.csv", encoding='cp1252')

df.columns = [col.strip().replace(" ", "_").replace("-", "_") for col in df.
      ↪columns]
orders_cols = [
    "Row_ID", "Order_ID", "Order_Date", "Ship_Date", "Ship_Mode",
    "Customer_ID", "Customer_Name", "Segment", "Country", "City",
    "State", "Postal_Code", "Region",
]
items_cols = [
    "Row_ID", "Product_ID", "Category", "Sub_Category",

```

```

    "Product_Name", "Sales", "Quantity", "Discount", "Profit",
]

df_orders = df[orders_cols].copy()
df_items = df[items_cols].copy()

df_orders["Order_Date"] = pd.to_datetime(df_orders["Order_Date"])
df_orders["Ship_Date"] = pd.to_datetime(df_orders["Ship_Date"])

```

```

[242]: print(df_orders.shape)
       print(df_items.shape)

```

```

(9994, 13)
(9994, 9)

```

```

[243]: # pg
pg_conn = pg.connect(**pg_creds)
table_ddl_pg = """
    CREATE TABLE IF NOT EXISTS superstore_orders (
        row_id          INTEGER PRIMARY KEY,
        order_id         VARCHAR(64),
        order_date       DATE,
        ship_date        DATE,
        ship_mode        VARCHAR(64),
        customer_id      VARCHAR(64),
        customer_name    VARCHAR(255),
        segment          VARCHAR(64),
        country          VARCHAR(128),
        city             VARCHAR(128),
        state            VARCHAR(128),
        postal_code      VARCHAR(32),
        region           VARCHAR(64)
    )
    """

insert_pg = """
    INSERT INTO superstore_orders (
        row_id, order_id, order_date, ship_date, ship_mode,
        customer_id, customer_name, segment, country, city,
        state, postal_code, region
    ) VALUES (
        %(Row_ID)s, %(Order_ID)s, %(Order_Date)s, %(Ship_Date)s, %(Ship_Mode)s,
        %(Customer_ID)s, %(Customer_Name)s, %(Segment)s, %(Country)s, %(City)s,
        %(State)s, %(Postal_Code)s, %(Region)s
    )
    ON CONFLICT (row_id) DO NOTHING
    """

```

```

with pg_conn.cursor() as curr:

    curr.execute(table_ddl_pg)
    pg_conn.commit()

    curr.executemany(insert_pg, df_orders.to_dict(orient="records"))
    print("          pg:", curr.rowcount)
    pg_conn.commit()

```

pg: 9994

```

[244]: with pg_conn.cursor() as curr:
        curr.execute('select count(*) from public.superstore_orders')
        print(curr.fetchall())

```

[(9994,)]

```

[245]: #          mysql
my_conn = pymysql.connect(**mysql_creds)
my_conn.autocommit(True)

my_table_ddl = """
    CREATE TABLE IF NOT EXISTS superstore_items (
        row_id          INT PRIMARY KEY,
        product_id       VARCHAR(64),
        category         VARCHAR(64),
        sub_category     VARCHAR(64),
        product_name     VARCHAR(255),
        sales            DOUBLE,
        quantity         INT,
        discount         DOUBLE,
        profit           DOUBLE
    )
    """

insert_my = """
    INSERT INTO superstore_items (
        row_id, product_id, category, sub_category,
        product_name, sales, quantity, discount, profit
    ) VALUES (
        %(Row_ID)s, %(Product_ID)s, %(Category)s, %(Sub_Category)s,
        %(Product_Name)s, %(Sales)s, %(Quantity)s, %(Discount)s, %(Profit)s
    )
    ON DUPLICATE KEY UPDATE
        product_id=VALUES(product_id),
        category=VALUES(category),
        sub_category=VALUES(sub_category),

```

```

        product_name=VALUES(product_name),
        sales=VALUES(sales),
        quantity=VALUES(quantity),
        discount=VALUES(discount),
        profit=VALUES(profit)
"""
with my_conn.cursor() as my_cur:
    my_cur.execute(my_table_ddl)
    my_conn.commit()

    my_cur.executemany(insert_my, df_items.to_dict(orient="records"))
    print("          MySQL:", my_cur.rowcount)
    my_conn.commit()

```

MySQL: 9994

```

[246]: with my_conn.cursor() as curr:
        curr.execute('select count(*) from superstore_items')
        print(curr.fetchall())

```

((9994,),)

```

[247]: conn_pg.close()
        my_conn.close()

```

```

[7]: #                               trino

trino_conn = connect(**trino_creds_upd)

with trino_conn.cursor() as trino_curr:
    trino_curr.execute("SHOW CATALOGS")
    catalogs = trino_curr.fetchall()
catalogs

```

```

[7]: [['iceberg'], ['minio'], ['mysql'], ['postgres'], ['system']]

,      -      minio/iceberg      Trino :)

```

```

[8]: trino_query = """
      SHOW TABLES FROM postgres.public
      """

with trino_conn.cursor() as trino_curr:
    trino_curr.execute(trino_query)
    print(trino_curr.fetchall())

```

```

[['iceberg_namespace_properties'], ['iceberg_namespaces'], ['iceberg_tables'],
['superstore_orders']]

```

0.1 1. sql - (pg)

```
[9]: pg_query = f"""
      SELECT
          date_trunc('month', o.order_date) AS order_month,
          COUNT(*) AS orders_cnt
      FROM postgres.public.superstore_orders AS o
      GROUP BY 1
      ORDER BY 2 DESC
      """

df_orders_by_date = pd.read_sql(pg_query, trino_conn)
df_orders_by_date.head()
```

C:\Users\Daniel\AppData\Local\Temp\ipykernel_28732\2696115135.py:10:

UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_orders_by_date = pd.read_sql(pg_query, trino_conn)
```

```
[9]: order_month  orders_cnt
0  2017-12-01      462
1  2017-09-01      459
2  2017-11-01      459
3  2016-11-01      370
4  2016-09-01      363
```

0.2 2. sql - (mysql)

```
[10]: msql_query = f"""
      SELECT
          i.category,
          ROUND(SUM(i.sales),2) AS total_sales,
          SUM(i.quantity) AS total_qty,
          ROUND(SUM(i.profit),2) AS total_profit
      FROM mysql.demo_db.superstore_items AS i
      GROUP BY i.category
      ORDER BY total_sales DESC
      """

df_sales_by_category = pd.read_sql(msql_query, trino_conn)
df_sales_by_category.head()
```

C:\Users\Daniel\AppData\Local\Temp\ipykernel_28732\3934009684.py:11:

UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_sales_by_category = pd.read_sql(msql_query, trino_conn)
```

```
[10]:
```

	category	total_sales	total_qty	total_profit
0	Technology	836154.03	6939	145454.95
1	Furniture	741999.80	8028	18451.27
2	Office Supplies	719047.03	22906	122490.80

0.3 3. sql - . (pg + mysql)

```
[11]: pg_mysql_query = f"""
SELECT
    o.region,
    o.segment,
    DATE_TRUNC('month', o.order_date) AS order_month,
    SUM(i.sales) AS total_sales,
    SUM(i.profit) AS total_profit,
    SUM(i.quantity) AS total_qty
FROM postgres.public.superstore_orders AS o
JOIN mysql.demo_db.superstore_items AS i
    ON o.row_id = i.row_id
GROUP BY
    o.region,
    o.segment,
    DATE_TRUNC('month', o.order_date)
ORDER BY
    o.region,
    o.segment,
    order_month
"""

df_region_segment_month = pd.read_sql(pg_mysql_query, trino_conn)
df_region_segment_month.head()
```

C:\Users\Daniel\AppData\Local\Temp\ipykernel_28732\2370166332.py:22:
UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_region_segment_month = pd.read_sql(pg_mysql_query, trino_conn)
```

```
[11]:
```

	region	segment	order_month	total_sales	total_profit	total_qty
0	Central	Consumer	2014-01-01	1099.206	139.7119	50
1	Central	Consumer	2014-02-01	1139.180	269.9888	56
2	Central	Consumer	2014-03-01	3877.690	-458.6878	91
3	Central	Consumer	2014-04-01	2835.652	-47.8489	61
4	Central	Consumer	2014-05-01	1726.586	-445.6287	74

```
[12]: # 1)
df_orders_by_date["metric_type"] = "orders_per_month"
df_orders_by_date_agg = df_orders_by_date.rename(columns={
    "order_month": "dimension",
```

```

        "orders_cnt": "metric_value"
    })[["dimension", "metric_value", "metric_type"]]

# 2)
df_sales_by_category["metric_type"] = "sales_per_category"
df_sales_by_category_agg = df_sales_by_category.rename(columns={
    "category": "dimension",
    "total_sales": "metric_value"
})[["dimension", "metric_value", "metric_type"]]

# 3)
df_region_segment_month["dimension"] = (
    df_region_segment_month["region"] + " | " +
    df_region_segment_month["segment"] + " | " +
    df_region_segment_month["order_month"].astype(str)
)
df_region_segment_month_agg = df_region_segment_month.rename(columns={
    "total_sales": "metric_value"
})[["dimension", "metric_value"]]
df_region_segment_month_agg["metric_type"] = "sales_region_segment_month"

#      DF
final_df = pd.concat(
    [df_orders_by_date_agg, df_sales_by_category_agg,
    ↪df_region_segment_month_agg],
    ignore_index=True
)
final_df.head()

```

```

[12]:   dimension  metric_value  metric_type
0  2017-12-01         462.0  orders_per_month
1  2017-09-01         459.0  orders_per_month
2  2017-11-01         459.0  orders_per_month
3  2016-11-01         370.0  orders_per_month
4  2016-09-01         363.0  orders_per_month

```

```

[281]: df_month = final_df[final_df["metric_type"] == "orders_per_month"].copy()
df_month = df_month.sort_values("dimension")
mean_val = df_month["metric_value"].mean()

plt.figure(figsize=(12, 5))

#
sns.lineplot(
    data=df_month,
    x="dimension",
    y="metric_value",

```

```

        color="steelblue",
        linewidth=2
    )

#
above_mean = df_month["metric_value"] > mean_val
plt.scatter(
    df_month.loc[~above_mean, "dimension"],
    df_month.loc[~above_mean, "metric_value"],
    color="steelblue",
    s=60,
    zorder=3,
    label="      "
)

plt.scatter(
    df_month.loc[above_mean, "dimension"],
    df_month.loc[above_mean, "metric_value"],
    color="orange",
    edgecolor="black",
    s=80,
    zorder=4,
    label=">      "
)

#
plt.axhline(mean_val, color="gray", linestyle="--", linewidth=1, alpha=0.7,
            label=f"      = {mean_val:.1f}")

plt.title("      ")
plt.xlabel("      ")
plt.ylabel("      ")
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```



```
[14]: df_cat = final_df[final_df["metric_type"] == "sales_per_category"].copy()
df_cat = df_cat.sort_values("metric_value", ascending=True) #

plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=df_cat,
    x="metric_value",
    y="dimension",
    palette="Blues_r"
)

# -
max_idx = df_cat["metric_value"].idxmax()
for i, (val, dim) in enumerate(zip(df_cat["metric_value"],
    ↪df_cat["dimension"])):
    if i == len(df_cat) - 1: # -
        ax.barh(i, val, color="orange", edgecolor="black")

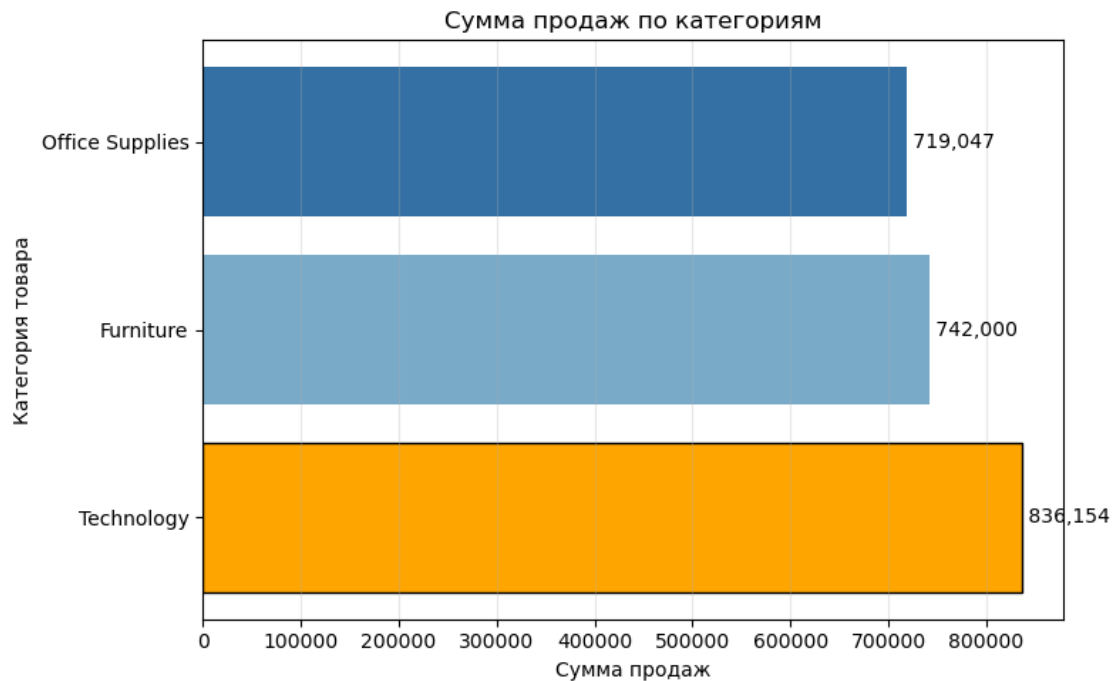
#
for i, v in enumerate(df_cat["metric_value"]):
    ax.text(v, i, f" {v:,.0f}", va="center")

plt.title(" ")
plt.xlabel(" ")
plt.ylabel(" ")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()
```

C:\Users\Daniel\AppData\Local\Temp\ipykernel_28732\1605601038.py:5:
FutureWarning:

Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0. Assign the ``y`` variable to ``hue`` and set ``legend=False`` for the same effect.

```
ax = sns.barplot(
```



0.4 Iceberg

```
[264]: from trino.dbapi import connect
```

```
hive_conn = connect(  
    host=trino_host,  
    port=trino_port,  
    user=trino_user,  
    catalog="iceberg",  
    schema="default",  
)  
  
with hive_conn.cursor() as cur:  
    cur.execute("CREATE SCHEMA IF NOT EXISTS iceberg.superstore")
```

```
[275]: #  
create_schema_sql = f"""  
    CREATE SCHEMA IF NOT EXISTS iceberg.superstore
```

```

"""
drop_if_exists = "DROP TABLE IF EXISTS iceberg.superstore.metrics_part"
create_table_sql = f"""
CREATE TABLE IF NOT EXISTS iceberg.superstore.metrics_part (
    dimension VARCHAR,
    metric_value DOUBLE,
    metric_type VARCHAR
)
WITH (
    format = 'PARQUET',
    partitioning = ARRAY['metric_type']
)
"""
#
with trino_conn.cursor() as cur_trino:
    cur_trino.execute(create_schema_sql)
    trino_conn.commit()
    cur_trino.execute(drop_if_exists)
    trino_conn.commit()
    cur_trino.execute(create_table_sql)
    trino_conn.commit()

```

```

[276]: #
rows = []
for row in final_df.itertuples(index=False):
    dim = str(row.dimension).replace("'", "'")
    metric_value = float(row.metric_value)
    metric_type = str(row.metric_type).replace("'", "'")
    rows.append(f"('{dim}', {metric_value}, '{metric_type}')"

insert_sql = f"""
INSERT INTO iceberg.superstore.metrics (dimension, metric_value, metric_type)
VALUES
{",\n".join(rows)}
"""

with trino_conn.cursor() as cur:
    cur.execute(insert_sql)
    trino_conn.commit()

```

```

[280]: #
check_sql = f"""
SELECT
    metric_type,
    COUNT(*) AS rows_cnt,
    MIN(metric_value) AS min_value,
    MAX(metric_value) AS max_value

```

```

FROM iceberg.superstore.metrics
GROUP BY metric_type
ORDER BY metric_type
"""
# dataquality - ,
df_check = pd.read_sql(check_sql, trino_conn)
df_check

```

C:\Users\Daniel\AppData\Local\Temp\ipykernel_43880\3844570892.py:13:

UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_check = pd.read_sql(check_sql, trino_conn)
```

```
[280]:
```

	metric_type	rows_cnt	min_value	max_value
0	orders_per_month	48	46.00	462.000
1	sales_per_category	3	719047.03	836154.030
2	sales_region_segment_month	560	9.64	25364.124

, pdf. iceberg trino minio.
!